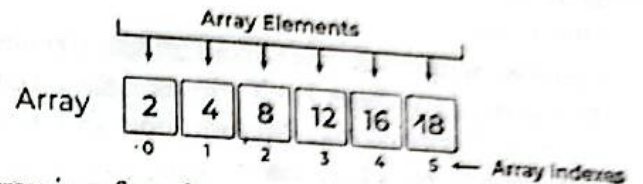


UNIT – 5

ARRAYS, STRINGS AND STRUCTURES**INTRODUCTION TO ARRAYS****5.1 Concept of an array and its Elements**

An array in C++ is a data structure that allows you to store multiple values of the same data type in a single, contiguous block of memory. Arrays are a fundamental concept in many programming languages, including C++, because they provide a way to manage collections of data efficiently. Each value stored in an array is referred to as an **element** of the array, and these elements are indexed, typically starting from zero.

**Declaring and Initializing Arrays:**

In C++, you declare an array by specifying the type of its elements, followed by the array name and the size (number of elements).

For example:

```
int numbers[5];
```

This declaration creates an array named `numbers` that can hold five integers. Each element in this array is of type `int`. Initially, the values of the array elements are undefined, so you should initialize them before using them. You can initialize an array at the time of declaration as follows:

```
int numbers[5] = {1, 2, 3, 4, 5};
```

This line of code initializes the array with five integers. The elements are stored in memory in the order they are declared, with the first element at index 0, the second at index 1, and so on.

Accessing Array Elements:

You access an element in an array by using the array name followed by the element's index in square brackets. For example, to access the first element of the `numbers` array, you would write:

```
int firstElement = numbers[0];
```

Here, `numbers[0]` refers to the first element in the array, which has the value 1. You can also modify an element by assigning a new value to it:

```
numbers[2] = 10;
```

This statement changes the third element of the `numbers` array (index 2) to 10.

Characteristics of Arrays:

One key characteristic of arrays in C++ is that they are **fixed in size**. Once an array is declared with a certain size, that size cannot be changed. This means that you must know the maximum number of elements your array will need to store when you declare it.

Arrays in C++ are also **zero-indexed**, meaning that the first element has an index of 0, the second element has an index of 1, and so on. This indexing allows you to use loops to iterate over the elements of the array. For example:

...this program, you can use any C++ compiler (such as GCC, Clang, or an IDE like Code::Blocks, Studio, or Xcode). Simply compile the code and execute the resulting binary to see the output.

Conclusion:

This program demonstrates how to access elements in a two-dimensional array using nested loops. Each element is printed along with its indices, showing how to navigate through a 2D array in C++. This approach is commonly used in applications that require processing data in a grid or matrix format, such as image processing, game development, and scientific computing.

STRINGS

5.9 Strings and Define a String

In C++, a string is a sequence of characters. Strings are widely used in programming to store and manipulate text. C++ provides two main ways to work with strings:

1. **Character Arrays:** Strings can be represented as arrays of characters, which is the traditional C-style string.
2. **std::string Class:** C++ provides a more convenient and powerful string handling through the `std::string` class, which is part of the Standard Template Library (STL).

1. Defining a String Using a Character Array (C-Style String)

A C-style string is an array of characters terminated by a null character (`\0`). The null character indicates the end of the string.

String in C++

C Style string

```
char e[] = "geeks"  
char e1[] = {'g', 'f', 'g', '\0'};  
char *c = "geeksforgeeks";
```

C++ Style string

```
string str = ("gfg");  
string str = "gfg";  
string str; str = "gfg";
```


Chapter – 5: Arrays, Strings and Structures

In this example, the age and height members of person1 are set directly, and the name is copied using strcpy().

4. Initializing Structures:

Structures can be initialized at the time of declaration.

Example:

```
struct Person person2 = {"Jane Doe", 28, 5.5};
```

Here, person2 is initialized with a name, age, and height.

5. Typedef with Structures:

Using typedef, you can create an alias for the structure type, simplifying its usage.

Syntax:

```
typedef struct StructureName {  
    dataType member1;  
    dataType member2;  
    ...  
} AliasName;
```

Example:

```
typedef struct Person {  
    char name[50];  
    int age;  
    float height;  
} Person;  
Person person3 = {"Alice", 25, 5.7};
```

Here, Person is an alias for struct Person, allowing you to declare structure variables without using the struct keyword.

6. Nested Structures:

Structures can contain other structures as members.

Example:

```
struct Date {  
    int day;  
    int month;  
    int year;  
};  
struct Employee {  
    char name[50];  
    struct Date dateOfJoining;  
    float salary;  
};
```

In this example, the Employee structure contains a Date structure as a member.

Conclusion:

Structures in C provide a powerful way to group related data together. They are foundational for building complex data types and are widely used in various applications, from simple data management to sophisticated data structures like linked lists, trees, and more.

```
substr(dest, src, 7, 5); // dest now contains "World"
```

Conclusion:

The functions `strlen()`, `strcat()`, and `strcpy()` are standard C library functions commonly used for string manipulation. The functions `strcmp()` and `substr()` are either non-standard or incorrectly named, with `substr()` typically being a custom implementation in C.

STRUCTURES

5.13 Structure and its Declaration

In C programming, a **struct** (short for "structure") is a user-defined data type that allows you to group different types of data together. Structures are useful for organizing complex data that involves multiple attributes or properties.

1. Structure Declaration

A structure is declared using the `struct` keyword, followed by the structure name and a block that defines its members (variables).

Syntax:

```
struct StructureName {  
    dataType member1;  
    dataType member2;  
    ...  
    dataType memberN;  
};
```

Example:

```
struct Person {  
    char name[50];  
    int age;  
    float height;  
};
```

In this example, a `Person` structure is defined with three members: `name` (a string), `age` (an integer), and `height` (a floating-point number).

2. Structure Variable Declaration

Once a structure is defined, you can create variables of that structure type.

Syntax:

```
struct StructureName variableName;
```

Example:

```
struct Person person1;
```

Here, `person1` is a variable of the `Person` structure type.

3. Accessing Structure Members:

Structure members are accessed using the dot operator (`.`).

Example:

```
person1.age = 30;  
person1.height = 5.9;  
strcpy(person1.name, "John Doe");
```